

Exercices d'application - Séance 3

Modalités : chaque étudiant traite individuellement les exercices. Les réponses sont rédigées dans le fichier `seance-03/RENDU.md` de votre fork (section « Exercices d'application »).

Exercice 1 : QCM conceptuel

Pour chaque question, choisissez **une seule** réponse, sauf indication contraire. Justifiez en une phrase.

1.1 Parmi ces affirmations sur Kubernetes, laquelle est vraie ?

- A. Kubernetes remplace Docker en fournissant son propre moteur de conteneurs.
- B. Kubernetes orchestre des conteneurs sur un cluster de machines, en s'appuyant sur un container runtime (containerd, Docker, CRI-O).
- C. Kubernetes ne peut tourner que sur des machines Linux nativement.
- D. Kubernetes est une alternative à la conteneurisation.

1.2 Quel composant du Control Plane stocke l'état complet du cluster ?

- A. API Server
- B. etcd
- C. Scheduler
- D. Controller Manager

1.3 Quel composant du Control Plane décide **sur quel nœud** un nouveau pod doit être placé ?

- A. API Server
- B. etcd
- C. Scheduler
- D. kubelet

1.4 Vous tapez `kubectl get pods` . À qui kubectl parle-t-il ?

- A. Directement aux pods.
- B. À etcd, qui contient la liste des pods.
- C. À l'API Server, qui est le point d'entrée unique du cluster.
- D. Au Scheduler, qui sait où sont placés les pods.

1.5 Vous supprimez un pod géré par un Deployment avec `kubectl delete pod <nom>` . Que se passe-t-il ?

- A. Le pod est définitivement supprimé.
- B. Le Deployment recrée immédiatement un nouveau pod pour respecter l'état souhaité.
- C. Tout le Deployment est supprimé en cascade.
- D. Une erreur est renvoyée car on ne peut pas supprimer un pod géré par un Deployment.

1.6 Quel type de Service permet l'accès à un Deployment **depuis l'extérieur du cluster** sans passer par un load balancer cloud ?

- A. ClusterIP
- B. NodePort
- C. LoadBalancer
- D. Ingress

1.7 Que fait la commande `kubectl scale deployment minio --replicas=5` ?

- A. Elle augmente la taille du cluster à 5 machines.
- B. Elle modifie l'état souhaité du Deployment à 5 replicas ; Kubernetes converge vers ce nombre.
- C. Elle crée 5 nouveaux Deployments.
- D. Elle ajoute 5 replicas en plus des replicas existants.

1.8 À quoi sert un **Namespace** Kubernetes ?

- A. À nommer le cluster.
- B. À isoler logiquement les ressources (séparation par équipe, environnement, ou application).
- C. À chiffrer les communications entre pods.
- D. À limiter les permissions des utilisateurs sur le cluster.

1.9 Avec **Kind**, les nœuds Kubernetes du cluster sont en réalité :

- A. Des machines virtuelles légères créées par Kind.
- B. Des conteneurs Docker.
- C. Des processus standalone sur l'hôte.
- D. Des Pods dans un cluster parent.

Exercice 2 : Lecture et interprétation d'un manifeste (*facile à moyen*)

Voici un manifeste Kubernetes qu'un étudiant a écrit pour déployer une API Python pour Anfa :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: anfa-api
  namespace: anfa
spec:
  replicas: 2
  selector:
    matchLabels:
      app: anfa-api
  template:
    metadata:
      labels:
        app: anfa-api
    spec:
      containers:
        - name: api
          image: anfa-api:v1.2
          ports:
            - containerPort: 8000
          env:
            - name: MINIO_ENDPOINT
              value: "http://minio:9000"
            - name: LOG_LEVEL
              value: "INFO"
```

Questions :

2.1 À quoi sert le champ `selector.matchLabels` , et quel est son lien avec `template.metadata.labels` ?

2.2 Combien de pods seront créés par ce Deployment ? Si l'un meurt, que fait Kubernetes ?

2.3 Le manifeste référence `http://minio:9000` comme endpoint MinIO. Pourquoi `minio` (et pas une adresse IP) ? Qu'est-ce qui rend cette résolution possible ?

2.4 Ce manifeste ne définit **aucun Service**. Quelle conséquence pratique cela a-t-il pour l'API ?

2.5 Proposez un manifeste de **Service** qui expose ce Deployment à l'intérieur du cluster uniquement, sur le port 80, redirigeant vers le port 8000 du conteneur.

Exercice 3 : Diagnostic

3.1 Le pod qui ne démarre pas

Un étudiant déploie un pod et observe ceci :

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
minio-7d9f8b6c5-x2k9p	0/1	ImagePullBackOff	0	2m

Son manifeste contient cette ligne :

```
image: minio/miniooo:latest
```

Questions :

- a. Que signifie le statut `ImagePullBackOff` ?
- b. Quelle est très probablement la cause ici ?
- c. Avec quelle commande peut-on obtenir plus de détails sur l'erreur ?

3.2 : Le PVC qui ne se lie pas

Un étudiant a appliqué ce manifeste de PVC :

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Gi
```

Il observe :

```
$ kubectl get pvc
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
data-pvc      Pending                                     standard       5m
```

Questions :

- a. Que signifie le statut `Pending` pour un PVC ?
- b. Quelle est la cause la plus probable dans ce contexte (cluster Kind local) ?
- c. Quelle commande utiliseriez-vous pour confirmer votre diagnostic ?

3.3 : Le port-forward qui échoue

Un étudiant lance :

```
$ kubectl port-forward service/minio 9001:9001  
error: unable to forward port because pod is not running. Current status=Pending
```

Questions :

- a. Pourquoi cette erreur apparaît-elle ?
- b. Quelle commande lancer pour comprendre pourquoi le pod est en `Pending` ?
- c. Quel est l'ordre logique à respecter avant de tenter un port-forward ?

Exercice 4 : De Docker Compose à Kubernetes

Voici un extrait du `docker-compose.yml` utilisé en séance 2 pour MinIO :

```
services:
  minio:
    image: minio/minio:latest
    container_name: anfa-minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: anfa-admin
      MINIO_ROOT_PASSWORD: anfa-password-2026
    volumes:
      - minio-data:/data
    command: server /data --console-address ":9001"

volumes:
  minio-data:
```

Questions :

4.1 Ce fichier décrit un seul service Compose. Combien de **manifestes Kubernetes distincts** seront nécessaires pour reproduire la même fonctionnalité ? Listez-les avec leur rôle.

4.2 En Docker Compose, `volumes: - minio-data:/data` suffit pour persister les données. En Kubernetes, la persistance est plus structurée. Expliquez en 2-3 lignes la différence conceptuelle entre un volume Docker nommé et un PersistentVolumeClaim Kubernetes.

4.3 En Compose, on accède à MinIO via `http://localhost:9001`. En Kubernetes avec un Service `NodePort`, on a vu qu'il fallait passer par un `kubectl port-forward` avec Kind. Pourquoi cette différence ? Que faudrait-il pour qu'on accède directement à MinIO sur un port de l'hôte, comme avec Compose ?

4.4 Citez **deux choses** que Kubernetes apporte par rapport à Docker Compose dans le cas de MinIO, et que vous avez observées concrètement en TP.

Exercice 5 : Mini-cas d'architecture

L'équipe d'Anfa veut industrialiser son architecture. Le contexte :

- Le pipeline batch (`pipeline-anfa`) doit tourner **chaque nuit à 2 h du matin** : il lit les positions GPS de la journée depuis MinIO, calcule des agrégats, et écrit les résultats dans MinIO. Durée : environ 15 minutes.
- Une **API REST** (`anfa-api`) expose les agrégats aux applications mobiles des conducteurs. Elle reçoit ~50 requêtes/seconde aux heures de pointe (matin et soir), ~5 requêtes/seconde le reste du temps. Elle doit être toujours disponible.
- Le **dashboard Grafana** (`anfa-dashboard`) est consulté par 3-4 chefs d'exploitation simultanément, en journée uniquement. Disponibilité standard.

Tout ce monde tourne dans un cluster Kubernetes managé chez un fournisseur cloud.

Questions :

5.1 Pour chacun des 3 composants (`pipeline-anfa` , `anfa-api` , `anfa-dashboard`), proposez le **type d'objet Kubernetes** le plus adapté parmi : `Deployment` , `CronJob` , `Job` , `StatefulSet` . Justifiez chaque choix en une phrase.

Note : `CronJob` n'a pas été détaillé en cours, mais devinez son rôle d'après son nom (cron = tâche planifiée Linux). `Job` exécute une tâche ponctuelle qui se termine. `StatefulSet` est utilisé pour les applications avec état persistant (bases de données).

5.2 Pour `anfa-api` , vous voulez exploiter l'**Horizontal Pod Autoscaler** vu en cours. Proposez les **paramètres** que vous choisiriez (`minReplicas` , `maxReplicas` , métrique cible). Justifiez en 2-3 lignes en lien avec le profil de charge décrit.

5.3 Pour `anfa-api`, quel type de **Service** choisiriez-vous parmi `ClusterIP`, `NodePort`, `LoadBalancer` ? Justifiez.

5.4 L'équipe craint qu'une mise à jour de `anfa-api` provoque une **coupure de service** pendant le redéploiement. Comment Kubernetes gère-t-il cela par défaut ? Expliquez en 3-4 lignes le mécanisme.

5.5 Proposez un **squelette de manifeste YAML** pour `anfa-api` (Deployment uniquement, ~20-30 lignes). N'écrivez pas l'HPA ni le Service : juste le Deployment avec 3 replicas, l'image `anfa/api:v1`, les ports, et une variable d'environnement `MINIO_ENDPOINT=http://minio:9000`.